NM i AI 2026 — Comprehensive Technical Cheat Sheet

Competition: March 19–22, 2026 | Four challenges: Grocery Bot · Race Car · Healthcare RAG · Tumor Segmentation

---

Quick Navigation

- Section 1: Grocery Bot — Multi-Agent Pathfinding
- Section 2: Race Car — Sensor-Based Navigation
- Section 3: RAG Systems — Emergency Healthcare
- Section 4: Medical Image Segmentation — Tumor in PET
- Section 5: Practical Competition Strategy
- Section 6: Quick Reference (Copy-Paste Ready)

SECTION 1: GROCERY BOT — Multi-Agent Pathfinding

1.1 A* Algorithm — Full Implementation

A* finds the optimal path by combining actual cost g(n) with a heuristic estimate h(n) to form f(n) = g(n) + h(n). For a grid-based grocery store, Manhattan distance is the go-to heuristic since movement is 4-directional (no diagonals).[1]

Core A* — Python (heapq-based, copy-paste ready)

```python
import heapq

def astar(grid, start, goal, blocked_cells=set()):
    """
    grid: 2D list of booleans (True = passable)
```

```
    start, goal: (row, col) tuples
    blocked_cells: set of (row, col) for other agents' current positions
    Returns: list of (row, col) or [] if no path
    """
    rows, cols = len(grid), len(grid)

    def heuristic(a, b):
        return abs(a - b) + abs(a[1] - b[1])

    open_set = []
    # (f, tie_break_h, g, node)
    heapq.heappush(open_set, (0 + heuristic(start, goal), heuristic(start, goal), 

    came_from = {}
    g_score = {start: 0}
    closed_set = set()

    while open_set:
        f, h, g, current = heapq.heappop(open_set)

        if current == goal:
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
            return path[::-1]

        if current in closed_set:
            continue
        closed_set.add(current)

        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
            nr, nc = current+dr, current[1]+dc
            neighbor = (nr, nc)
            if not (0 <= nr < rows and 0 <= nc < cols):
                continue
            if not grid[nr][nc]:
```

```
            continue
        if neighbor in blocked_cells and neighbor != goal:
            continue

        tentative_g = g + 1
        if tentative_g < g_score.get(neighbor, float('inf')):
            came_from[neighbor] = current
            g_score[neighbor] = tentative_g
            h_val = heuristic(neighbor, goal)
            # Tie-breaking: prefer nodes closer to goal
            heapq.heappush(open_set, (tentative_g + h_val, h_val, tentative_g, n

    return []  # No path found
```

Key A* concepts for this competition

Tie-breaking: When multiple nodes have the same f(n), break ties by preferring lower h(n) (nodes closer to the goal). This dramatically reduces explored nodes on open grids and speeds up computation. The tuple `(f, h, g, node)` in the heap handles this automatically.[1]

Performance tips:

- Use `set` for `closed_set` — O(1) lookup
- For a 30×18 = 540 cell grid, even pure Python A* runs in microseconds
- Cache paths: if the grid hasn't changed, reuse paths; only replan on collision or obstacle change
- Precompute the grid as a 2D boolean array, not dict lookups

Common A* Mistakes

- Forgetting to check `if current in closed_set` after popping (stale entries in heap)
- Using `list.remove()` on the open set — O(n), use `heapq` instead
- Not handling ties — causes slow, winding paths on open grids
- Re-planning every single tick — expensive; only replan when blocked

---

1.2 Multi-Agent Coordination: Cooperative A* (HCA*/CA*)

The classic approach for multi-bot grid pathfinding is **Hierarchical Cooperative A*** (HCA*) by David Silver:[2]

1. Assign a **priority ordering** to agents (e.g., by distance to goal or by agent ID)
2. Plan path for agent 1 with no constraints
3. Plan path for agent 2, treating agent 1's planned positions as blocked at each timestep
4. Continue in order — each agent avoids all previously planned agents
5. Store all planned positions in a **reservation table** (3D: row × col × time)

```
from collections import defaultdict

class ReservationTable:
    def __init__(self):
        # Maps (row, col, timestep) -> agent_id
        self.reserved = {}

    def reserve(self, path, agent_id):
        for t, pos in enumerate(path):
            self.reserved[(pos, pos[1], t)] = agent_id
        # Also reserve final position indefinitely (agent waits)
        if path:
            final = path[-1]
            for t in range(len(path), len(path) + 50):
                self.reserved[(final, final[1], t)] = agent_id

    def is_blocked(self, row, col, t):
        return (row, col, t) in self.reserved

def astar_spacetime(grid, start, goal, reservation_table, max_t=100):
    """A* in (row, col, time) space to avoid other agents."""
    rows, cols = len(grid), len(grid)
```

```python
def h(pos):
    return abs(pos-goal) + abs(pos[1]-goal[1])

open_set = [(h(start), 0, start, 0)]  # (f, g, pos, t)
came_from = {}
visited = set()

while open_set:
    f, g, pos, t = heapq.heappop(open_set)
    state = (pos, t)
    if state in visited:
        continue
    visited.add(state)

    if pos == goal:
        path = []
        while (pos, t) in came_from:
            path.append(pos)
            pos, t = came_from[(pos, t)]
        path.append(start)
        return path[::-1]

    if t >= max_t:
        continue

    # Try move + wait
    for dr, dc in [(-1,0),(1,0),(0,-1),(0,1),(0,0)]:
        nr, nc = pos+dr, pos[1]+dc
        next_pos = (nr, nc)
        if not (0 <= nr < rows and 0 <= nc < cols):
            continue
        if not grid[nr][nc]:
            continue
        if reservation_table.is_blocked(nr, nc, t+1):
            continue
        new_g = g + 1
        came_from[(next_pos, t+1)] = (pos, t)
```

```
            heapq.heappush(open_set, (new_g + h(next_pos), new_g, next_pos, t+1

    return []

def plan_all_agents(grid, starts, goals):
    table = ReservationTable()
    all_paths = []
    # Sort by distance to goal (priority: shorter paths first)
    order = sorted(range(len(starts)), key=lambda i: abs(starts[i]-goals[i])+abs(s

    paths = [None] * len(starts)
    for i in order:
        path = astar_spacetime(grid, starts[i], goals[i], table)
        table.reserve(path, i)
        paths[i] = path
    return paths
```

Deadlock Prevention[3][4]

Deadlocks occur when agents form a cycle of mutual waiting. Key strategies:

- **Wait action**: Include `(0, 0)` as a valid move in space-time A* — agents can pause in place
- **Priority ordering with re-planning**: If agent A is blocked by agent B, temporarily boost B's priority and replan
- **PIBT (Priority Inheritance with Backtracking)**: Each timestep, agents in priority order claim their next cell; if a cell is claimed, the lower-priority agent yields. Efficient for 20+ agents[5]
- **Simple fallback**: If stuck for N ticks, take a random valid step to break symmetry

Practical deadlock fix for competition

```
def get_next_move(path, current_pos, tick, stuck_counter):
    if not path or path == current_pos:
        if len(path) > 0:
            path.pop(0)
```

```
        return current_pos  # Wait

    next_pos = path

    # If stuck for 3+ ticks, try random move
    if stuck_counter >= 3:
        import random
        neighbors = [(current_pos+dr, current_pos[1]+dc)
                     for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]]
        valid = [p for p in neighbors if is_valid(p)]
        if valid:
            return random.choice(valid)

    return next_pos
```

1.3 Order Batching and Item Pickup Strategy

Scoring recap

- +1 per item delivered to drop zone
- +5 per completed order
- Max capacity: 3 items per bot
- Priority: **complete orders for +5 bonus** over random item collection

Batching strategy

Group items by spatial proximity:[6]

```
from itertools import combinations

def batch_orders(orders, max_items=3):
    """
    orders: list of dicts with 'items' (list of item locations) and 'order_id'
    Returns: list of batches, each batch = list of item locations to pick
    """
```

```
    batches = []
    pending_items = []

    # Flatten orders into (item_location, order_id) pairs
    for order in orders:
        for item_loc in order['items']:
            pending_items.append({'loc': item_loc, 'order_id': order['order_id']})

    # Greedy: pick items that complete orders first
    # Sort by order completion potential
    order_progress = {}  # order_id -> items already assigned
    for order in orders:
        order_progress[order['order_id']] = 0

    current_batch = []
    for item in pending_items:
        if len(current_batch) < max_items:
            current_batch.append(item)
            order_progress[item['order_id']] += 1
        else:
            batches.append(current_batch)
            current_batch = [item]
            order_progress[item['order_id']] = 1

    if current_batch:
        batches.append(current_batch)

    return batches

def nearest_item_order(bot_pos, items):
    """Return items sorted by distance from bot position."""
    return sorted(items, key=lambda x: abs(x['loc']-bot_pos)+abs(x['loc'][1]-bo
```

Key inventory rules

- **Always pick up items that complete an order first** — the +5 bonus outweighs travel cost
- With 3-item capacity, aim to carry items that collectively complete 1 order per trip
- If no order can be completed in this trip, pick the 3 nearest items
- **Never go to the drop zone with 1 item** unless all other items require a detour > 5 cells

---

1.4 WebSocket Async Programming

Robust async game loop template[7][8]

```
import asyncio
import json
import websockets
from websockets.exceptions import ConnectionClosed

GAME_WS_URL = "ws://your-game-server/ws"

class GameBot:
    def __init__(self, url):
        self.url = url
        self.ws = None
        self.running = True

    async def connect_and_play(self):
        while self.running:
            try:
                async with websockets.connect(
                    self.url,
                    ping_interval=20,
                    ping_timeout=10,
                    open_timeout=10,
                ) as ws:
                    self.ws = ws
                    print(f"Connected to {self.url}")
```

```
            await self.game_loop(ws)
        except ConnectionClosed as e:
            print(f"Connection closed: {e.code} {e.reason}")
            await asyncio.sleep(1)
        except OSError as e:
            print(f"Network error: {e}")
            await asyncio.sleep(2)
        except Exception as e:
            print(f"Unexpected error: {e}")
            await asyncio.sleep(1)

    async def game_loop(self, ws):
        async for raw_message in ws:
            try:
                state = json.loads(raw_message)
                # MUST respond within 2 seconds
                action = await asyncio.wait_for(
                    self.compute_action(state),
                    timeout=1.8  # leave 200ms buffer
                )
                await ws.send(json.dumps(action))
            except asyncio.TimeoutError:
                # Send a safe default action if computation times out
                await ws.send(json.dumps(self.default_action()))
            except Exception as e:
                print(f"Error processing state: {e}")
                await ws.send(json.dumps(self.default_action()))

    async def compute_action(self, state):
        """Override this with your bot logic."""
        # Parse game state
        bots = state.get('bots', [])
        orders = state.get('orders', [])
        grid = state.get('grid', [])

        actions = {}
        for bot in bots:
            bot_id = bot['id']
```

```
            bot_pos = (bot['row'], bot['col'])
            target = self.decide_target(bot, orders, grid)
            path = astar(grid, bot_pos, target)
            if path and len(path) > 1:
                actions[bot_id] = path[1]  # next step
            else:
                actions[bot_id] = bot_pos  # wait

        return {'actions': actions}

    def default_action(self):
        return {'actions': {}}

    def decide_target(self, bot, orders, grid):
        # Implement your targeting logic here
        pass

async def main():
    bot = GameBot(GAME_WS_URL)
    await bot.connect_and_play()

if __name__ == "__main__":
    asyncio.run(main())
```

Parsing game state efficiently

```
def parse_game_state(raw):
    state = json.loads(raw)

    # Convert grid to 2D boolean array once
    grid_raw = state['grid']
    grid = [[cell != 'WALL' for cell in row] for row in grid_raw]

    # Index shelves by position for O(1) lookup
    shelves = {}
    for shelf in state.get('shelves', []):
        shelves[(shelf['row'], shelf['col'])] = shelf['items']
```

```
    # Index orders by order_id
    orders = {o['id']: o for o in state.get('orders', [])}

    # Current bot states
    bots = {b['id']: b for b in state.get('bots', [])}

    return grid, shelves, orders, bots
```

2-second timeout management

- Precompute paths when possible; cache if grid/state hasn't changed
- Limit A* iterations: add an `iteration_limit` parameter
- With 20 bots on 30×18: run A* serially; Python is fast enough for 20 calls on this grid size
- If time budget is tight, **reduce path length**: plan only 5-10 steps ahead and replan next tick

---

1.5 Scaling from 1 Bot (Easy) to 20 Bots (Nightmare)

Difficulty	Bots	Grid	Strategy
Easy	1	12×10	Pure A* to nearest order item
Medium	5	18×14	A* + basic collision (yield on conflict)
Hard	10	22×16	CA*/HCA* with reservation table
Expert	15	26×18	CA* + deadlock detection
Nightmare	20	30×18	CA* + PIBT fallback + order batching

Assignment algorithm for 20 bots:

```python
from scipy.optimize import linear_sum_assignment
import numpy as np

def assign_bots_to_tasks(bots, tasks):
    """
    bots: list of (row, col)
    tasks: list of (row, col) target positions
    Returns: list of (bot_idx, task_idx) assignments
    """
    n_bots = len(bots)
    n_tasks = len(tasks)

    # Build cost matrix (Manhattan distance)
    size = max(n_bots, n_tasks)
    cost = np.full((size, size), 9999)
    for i, b in enumerate(bots):
        for j, t in enumerate(tasks):
            cost[i][j] = abs(b-t) + abs(b[1]-t[1])

    row_ind, col_ind = linear_sum_assignment(cost)
    return [(r, c) for r, c in zip(row_ind, col_ind) if r < n_bots and c < n_tasks]
```

Common Pitfalls in Grid-Based Game AI

- **Off-by-one in grid indexing**: Always confirm whether grid is row-major and what (0,0) represents
- **Blocking yourself**: A bot waiting on a cell blocks others — implement the (0,0) wait action
- **Thrashing**: Two bots trying to swap positions deadlock — detect and resolve with priority
- **Stale paths**: Always check if the path target is still valid (item may have been picked up)
- **Forgetting drop zone capacity**: If drop zone is full/unreachable, handle gracefully
- **JSON serialization overhead**: Pre-encode your action dict, don't rebuild it every tick

SECTION 2: RACE CAR — Sensor-Based Navigation

2.1 Processing 16 Directional Sensors

The car has 16 sensors measuring distances to obstacles in 16 directions (likely evenly spaced at 22.5° intervals). The sensor vector is your full state.[9]

```
import numpy as np

def parse_sensors(sensor_values):
    """
    sensor_values: list of 16 floats (distances, 0=wall, large=open)
    Returns: structured sensor readings
    """
    n = len(sensor_values)
    angles = [i * (360 / n) for i in range(n)]

    # Find forward-facing sensors (front 90 degrees = sensors 0, 1, 15, 14)
    front_sensors = sensor_values[0:3] + [sensor_values[-1], sensor_values[-2]]

    # Lateral sensors
    left_sensors  = sensor_values[3:7]   # left side
    right_sensors = sensor_values[9:13]  # right side

    # Key derived features
    forward_clearance = min(front_sensors) if front_sensors else 0
    left_clearance    = min(left_sensors) if left_sensors else 0
    right_clearance   = min(right_sensors) if right_sensors else 0

    return {
        'forward': forward_clearance,
        'left': left_clearance,
        'right': right_clearance,
```

```
        'raw': sensor_values,
        'max_direction': np.argmax(sensor_values),  # direction with most clearan
    }
```

2.2 PID Controller (Recommended First Approach)

For a competition with time pressure, start with a rule-based or PID approach. It takes 15 minutes to implement, runs at 1000 Hz, and can score competitively without training time.[10]

PID implementation

```
class PIDController:
    def __init__(self, kp, ki, kd, setpoint=0.0):
        self.kp = kp
        self.ki = ki
        self.kd = kd
        self.setpoint = setpoint
        self.prev_error = 0.0
        self.integral = 0.0
        self.dt = 1.0  # normalize; tune based on actual tick rate

    def update(self, measured_value):
        error = self.setpoint - measured_value
        self.integral += error * self.dt
        derivative = (error - self.prev_error) / self.dt
        output = self.kp * error + self.ki * self.integral + self.kd * derivative
        self.prev_error = error
        return output

# Steering PID: keep left/right sensor balance = 0
steering_pid = PIDController(kp=0.5, ki=0.01, kd=0.1, setpoint=0.0)

def compute_action(sensors):
    s = parse_sensors(sensors)
```

```
    # Steering: balance left/right clearance
    balance = s['left'] - s['right']
    steering = steering_pid.update(-balance)  # negative feedback
    steering = max(-1.0, min(1.0, steering))

    # Throttle: go faster when forward is clear, brake when walls close
    if s['forward'] > 50:
        throttle = 1.0
    elif s['forward'] > 20:
        throttle = 0.6
    elif s['forward'] > 10:
        throttle = 0.3
    else:
        throttle = 0.0  # brake / reverse

    return {'steering': steering, 'throttle': throttle}
```

PID Tuning Guide

Parameter	Effect	Tune when
Kp (Proportional)	Immediate response	Car reacts too slowly or oscillates
Ki (Integral)	Eliminates steady-state drift	Car consistently drifts to one side
Kd (Derivative)	Damps oscillation	Car overshoots and oscillates

Starting values: Kp=0.5, Ki=0.0, Kd=0.1. Increase Kp until oscillation, then add Kd to damp.

2.3 Rule-Based Agent (Fastest to Implement)

```python
def rule_based_action(sensors):
    """
    Pure rule-based: no training needed.
    Run this first to establish a baseline.
    """
    s = parse_sensors(sensors)
    raw = s['raw']

    # Find the direction with most clearance
    best_dir = int(np.argmax(raw))
    n = len(raw)

    # Map direction index to steering (-1 = left, 1 = right)
    # Assuming sensor = forward, going clockwise
    if best_dir <= n // 4:  # front-right quadrant
        steering = best_dir / (n / 4)
    elif best_dir >= 3 * n // 4:  # front-left quadrant
        steering = -(n - best_dir) / (n / 4)
    else:  # behind — reverse slightly
        steering = 1.0  # turn hard

    # Emergency brake
    if s['forward'] < 5:
        throttle = -0.3
    elif s['forward'] < 15:
        throttle = 0.2
    else:
        throttle = min(1.0, s['forward'] / 50)

    return {'steering': float(np.clip(steering, -1, 1)), 'throttle': float(throttle)}
```

2.4 Reinforcement Learning with PPO (If You Have Time)

Use PPO (Proximal Policy Optimization) via Stable Baselines3 if you have 1-2 hours to train. PPO is the go-to for continuous control tasks like racing.[11][12][9]

```
# pip install stable-baselines3 gymnasium
import gymnasium as gym
from stable_baselines3 import PPO
from stable_baselines3.common.env_checker import check_env
from stable_baselines3.common.vec_env import DummyVecEnv, VecNormal

# Wrap your competition environment in a Gym interface
class RaceCarEnv(gym.Env):
    def __init__(self, game_api):
        super().__init__()
        self.game_api = game_api
        # 16 sensors, normalized to [0, 1]
        self.observation_space = gym.spaces.Box(
            low=0.0, high=1.0, shape=(16,), dtype=np.float32
        )
        # steering + throttle, each in [-1, 1]
        self.action_space = gym.spaces.Box(
            low=np.array([-1.0, -1.0]),
            high=np.array([1.0, 1.0]),
            dtype=np.float32
        )
        self.max_steps = 3600  # 60 seconds at 60 Hz
        self.step_count = 0
        self.distance = 0.0

    def reset(self, seed=None):
        self.step_count = 0
        self.distance = 0.0
        obs = self.game_api.reset()
        return np.array(obs['sensors'], dtype=np.float32), {}
```

```python
    def step(self, action):
        steering, throttle = float(action), float(action[1])
        result = self.game_api.step({'steering': steering, 'throttle': throttle})

        obs = np.array(result['sensors'], dtype=np.float32)
        crashed = result.get('crashed', False)
        delta_dist = result.get('distance_delta', 0.0)
        self.distance += delta_dist
        self.step_count += 1

        # Reward shaping
        reward = delta_dist  # distance traveled this step
        if crashed:
            reward -= 100.0   # heavy crash penalty

        terminated = crashed
        truncated = self.step_count >= self.max_steps
        return obs, reward, terminated, truncated, {}

# Training
env = DummyVecEnv([lambda: RaceCarEnv(your_api)])
env = VecNormalize(env, norm_obs=True, norm_reward=True)

model = PPO(
    "MlpPolicy", env,
    n_steps=2048,
    batch_size=64,
    n_epochs=10,
    gamma=0.99,
    gae_lambda=0.95,
    clip_range=0.2,
    learning_rate=3e-4,
    verbose=1
)
```

```
model.learn(total_timesteps=500_000)  # ~1-2 hours depending on sim speed
model.save("race_car_ppo")
```

When to use RL vs. Rule-based

Approach	Pros	Cons	Use When
Rule-based	Instant, no training	Fragile, misses complex situations	First 30 min, baseline
PID	Fast to tune, smooth	Needs manual tuning	Always try before RL
PPO	Near-optimal, adapts	Needs training time + API	Have 1+ hours to train
SAC	Sample-efficient off-policy	More complex to set up	If PPO not converging

Competition recommendation: Start with rule-based → add PID steering → if time allows, train PPO for 500k steps. PPO with `MlpPolicy` on 16 sensor inputs trains fast.[13]

Reward Shaping for Distance Maximization

```
# Key reward signals
reward = delta_distance        # Primary: reward forward progress
reward -= 0.1 * abs(steering)  # Small penalty for sharp turns (encourages sm
reward -= 50 * int(crashed)    # Large crash penalty
# Optional: bonus for speed
reward += 0.01 * current_speed
```

Common Mistakes

- **Forgetting to normalize sensors**: Scale all sensor readings to before feeding to RL[14]
- **Too large crash penalty**: Can cause the agent to go very slowly to avoid crashes — balance it
- **No action smoothing**: Clipping actions to [-1, 1] still allows jerky movements; add `action = 0.5 * prev_action + 0.5 * new_action`
- **Single environment training**: Use `DummyVecEnv` or `SubprocVecEnv` with 4+ parallel envs to speed up PPO

SECTION 3: RAG SYSTEMS — Emergency Healthcare

3.1 RAG Architecture End-to-End

A complete RAG pipeline has two phases:[15]

Offline (pre-compute once):

```
Medical documents → Chunking → Embedding → FAISS index (saved to disk
+ Topic labels for each chunk → Label lookup table
```

Online (per query, must be <500ms):

```
Statement → Embed query → FAISS search (top-k chunks) → LLM/classifier -
```

```
[Query Statement]
      │
      ▼
[Embedding Model]  ←── ~10-50ms (MiniLM-L6 on CPU)
      │
      ▼
[FAISS IndexFlatIP]  ←── ~1ms (small corpus)
      │
```

```
    ▼
[Retrieved top-5 chunks + metadata]
    │
    ├──→ [True/False Classifier]  ← NLI model or LLM prompt
    └──→ [Topic Classifier]       ← cosine sim to topic centroids
```

3.2 Best Embedding Models for This Task

Model	Dims	Speed (CPU)	Size	Best For
`all-MiniLM-L6-v2`	384	~0.5ms/sentence	22M params	**First choice for speed**
all-mpnet-base-v2	768	~2ms/sentence	110M params	Higher accuracy, 4x slower
`paraphrase-multilingual-MiniLM-L12-v2`	384	~1ms/sentence	118M params	If Norwegian text present
text-embedding-3-small (OpenAI API)	1536	~50ms (network)	API-only	High accuracy, needs API key

Competition recommendation: `all-MiniLM-L6-v2` — ~22M parameters, ~0.5ms latency per sentence, ~84% STS accuracy. Achieves ~2,000 embeddings/second on GPU.[16][17]

```python
# pip install sentence-transformers faiss-cpu
from sentence_transformers import SentenceTransformer
import numpy as np

model = SentenceTransformer('all-MiniLM-L6-v2')

# Batch embed your corpus (do this once, offline)
```

```python
def embed_corpus(texts, batch_size=256):
    embeddings = model.encode(
        texts,
        batch_size=batch_size,
        normalize_embeddings=True,  # Important for cosine similarity with Inc
        show_progress_bar=True,
        convert_to_numpy=True
    )
    return embeddings.astype(np.float32)

# Single query embedding (online)
def embed_query(text):
    return model.encode(
        [text],
        normalize_embeddings=True,
        convert_to_numpy=True
    ).astype(np.float32)
```

3.3 FAISS Index — Which to Use

Index	When to Use	Speed	Recall	Memory
`IndexFlatIP`	< 100k vectors, need perfect recall	Slow	1.0	High
`IndexFlatL2`	< 100k vectors, L2 distance	Slow	1.0	High
`IndexIVFFlat`	50k-10M vectors, need speed	Fast	0.95	Medium
`IndexHNSWFlat`	Any size, best speed/recall balance	Very Fast	0.95+	High

For 115-topic medical corpus (likely < 50k chunks): **use `IndexFlatIP`** — perfect recall, simple, fast enough.[18][19]

```python
import faiss

def build_faiss_index(embeddings, use_gpu=False):
    """
    embeddings: np.float32 array of shape (n_docs, dim)
    Assumes embeddings are L2-normalized (use normalize_embeddings=Tru
    """
    dim = embeddings.shape[1]

    # For small corpus (<100k): IndexFlatIP (inner product = cosine sim when no
    index = faiss.IndexFlatIP(dim)

    # For larger corpus, use HNSW:
    # index = faiss.IndexHNSWFlat(dim, 32)  # 32 = M parameter
    # index.hnsw.efConstruction = 64
    # index.hnsw.efSearch = 32

    index.add(embeddings)

    if use_gpu and faiss.get_num_gpus() > 0:
        res = faiss.StandardGpuResources()
        index = faiss.index_cpu_to_gpu(res, 0, index)

    return index

def search(index, query_embedding, k=5):
    """Returns (distances, indices) — distances are cosine similarities"""
    scores, indices = index.search(query_embedding, k)
    return scores, indices

# Save/load index
faiss.write_index(index, "medical_index.faiss")
loaded_index = faiss.read_index("medical_index.faiss")
```

3.4 True/False Statement Verification

Method 1: NLI (Natural Language Inference) — Fastest

Use a cross-encoder NLI model. It scores whether context ENTAILS, CONTRADICTS, or is NEUTRAL to the statement.[20][21]

```
from transformers import pipeline

# Load once at startup
nli_pipeline = pipeline(
    "zero-shot-classification",
    model="cross-encoder/nli-distilroberta-base",
    device=0 if torch.cuda.is_available() else -1
)

def verify_statement(statement, retrieved_chunks):
    """
    Returns: {'label': 'true'/'false'/'uncertain', 'confidence': float}
    """
    context = " ".join(retrieved_chunks[:3])  # Use top-3 chunks

    result = nli_pipeline(
        sequences=f"Context: {context} Statement: {statement}",
        candidate_labels=["true", "false"]
    )

    label = result['labels']
    confidence = result['scores']
    return {'label': label, 'confidence': confidence}
```

Method 2: LLM Prompt (Higher accuracy, slower)

```
def verify_with_llm(statement, chunks, client):
    """Use when you have OpenAI/local LLM access."""
    context = "\n".join([f"[{i+1}] {c}" for i, c in enumerate(chunks[:5])])
```

```
    prompt = f"""You are a medical fact-checker.

Context documents:
{context}

Statement to verify: "{statement}"

Answer with ONLY "TRUE" or "FALSE" based on the context above.
If the context doesn't clearly support or contradict the statement, answer "FA

    response = client.chat.completions.create(
        model="gpt-4o-mini",  # fast + cheap
        messages=[{"role": "user", "content": prompt}],
        max_tokens=10,
        temperature=0
    )
    answer = response.choices.message.content.strip().upper()
    return answer in ["TRUE", "T", "YES"]
```

3.5 Topic Classification: 115 Classes

Zero-shot approach (no fine-tuning needed)

The fastest approach: pre-embed all 115 topic labels, then classify by cosine similarity to query.[22]

```
# Pre-compute topic embeddings ONCE at startup
TOPICS = ["cardiovascular disease", "diabetes management", "oncology treatr

topic_embeddings = model.encode(TOPICS, normalize_embeddings=True).as
topic_index = faiss.IndexFlatIP(topic_embeddings.shape[1])
topic_index.add(topic_embeddings)

def classify_topic(statement):
    """Returns topic index (0-114) and confidence score."""
    query_emb = embed_query(statement)
```

```
    scores, indices = topic_index.search(query_emb, 1)
    return int(indices), float(scores)
```

Fine-tuned approach (higher accuracy, needs labeled data)

If you have labeled statement→topic pairs, fine-tune a small classifier:

```
from sentence_transformers import SentenceTransformer
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder
import joblib

# One-time training (fast: 22M param model + linear head)
def train_topic_classifier(statements, labels):
    embs = model.encode(statements, normalize_embeddings=True)
    le = LabelEncoder()
    y = le.fit_transform(labels)
    clf = LogisticRegression(max_iter=1000, C=1.0)
    clf.fit(embs, y)
    return clf, le

# Inference
def predict_topic_trained(statement, clf, le):
    emb = embed_query(statement)
    pred = clf.predict(emb)
    prob = clf.predict_proba(emb).max()
    return le.inverse_transform([pred]), prob
```

3.6 Full Pipeline: Query to Answer in < 500ms

```
import time
import faiss
```

```python
import numpy as np
from sentence_transformers import SentenceTransformer

class MedicalRAGSystem:
    def __init__(self, corpus_path, index_path):
        # Load all at startup — this is the slow part (1-5s)
        self.model = SentenceTransformer('all-MiniLM-L6-v2')
        self.chunks = self._load_corpus(corpus_path)
        self.index = faiss.read_index(index_path)

        # Pre-embed topic labels
        self.topics = self._load_topics()  # list of 115 topic strings
        topic_embs = self.model.encode(self.topics, normalize_embeddings=Tru
        self.topic_index = faiss.IndexFlatIP(topic_embs.shape[1])
        self.topic_index.add(topic_embs)

        # Cache: statement → result (avoid recomputing identical queries)
        self.cache = {}

    def _load_corpus(self, path):
        import json
        with open(path) as f:
            return json.load(f)  # list of {"text": ..., "metadata": ...}

    def _load_topics(self):
        # Return list of 115 topic strings
        pass

    def answer(self, statement: str) -> dict:
        t0 = time.time()

        # Cache hit
        if statement in self.cache:
            return self.cache[statement]

        # 1. Embed query (~10ms)
        query_emb = self.model.encode([statement], normalize_embeddings=Tru
```

```
        # 2. Retrieve top-5 chunks (~1ms)
        scores, indices = self.index.search(query_emb, 5)
        retrieved = [self.chunks[i] for i in indices if i >= 0]

        # 3. Classify topic via similarity (~0.5ms)
        topic_scores, topic_indices = self.topic_index.search(query_emb, 1)
        topic_id = int(topic_indices)

        # 4. Verify true/false using top chunk (~depends on method)
        # Fast method: check if statement text is highly similar to retrieved chunk
        is_true = self._verify(statement, retrieved, scores)

        result = {
            'statement': statement,
            'is_true': is_true,
            'topic_id': topic_id,
            'topic': self.topics[topic_id],
            'latency_ms': (time.time() - t0) * 1000,
            'supporting_chunks': [r['text'] for r in retrieved[:2]]
        }
        self.cache[statement] = result
        return result

    def _verify(self, statement, retrieved_chunks, scores):
        """Fast heuristic: high retrieval similarity → likely true."""
        if not retrieved_chunks:
            return False
        top_score = float(scores)
        # If the statement is very similar to a retrieved chunk, it's likely true
        # Tune this threshold on your validation set
        return top_score > 0.75
```

Speed optimization checklist

- [ ] Load model and index once at startup, not per request
- [ ] Use `normalize_embeddings=True` to avoid separate normalization step
- [ ] Pre-embed all topic labels at startup

- [ ] Use `IndexFlatIP` for cosine similarity (faster than L2 + normalization)
- [ ] Implement a dict-based response cache
- [ ] Use `batch_size=256` for corpus embedding
- [ ] If using OpenAI API, use `gpt-4o-mini` with `max_tokens=5` and `temperature=0`

Norwegian medical terminology considerations

- The `paraphrase-multilingual-MiniLM-L12-v2` model handles Norwegian text[17]
- For a mixed Norwegian/English corpus, embed statements and chunks with the multilingual model
- Medical terminology tends to be similar across languages (Latin-origin terms)
- If Norwegian terms are used, include both Norwegian and English versions in your topic label strings

Common Mistakes

- **Chunking too large**: Chunks > 512 tokens reduce retrieval precision — use 200-300 tokens with 50-token overlap
- **Not normalizing embeddings**: Using `IndexFlatIP` requires L2-normalized vectors; without normalization, results are dot product, not cosine similarity
- **Cold start**: Loading model takes 3-5s; ensure it's loaded before the first query
- **No caching**: In a competition, many statements may be identical or very similar — always cache

---

SECTION 4: MEDICAL IMAGE SEGMENTATION — Tumor in PET

4.1 MIP-PET Images: What They Are and How to Load

MIP = Maximum Intensity Projection. A PET image is a 3D volume; a MIP projects it into 2D by taking the maximum intensity value along one axis (typically the anterior-posterior axis). This is how radiologists first assess tumor location.[23][24]

Key facts:

- PET images measure FDG (fluorodeoxyglucose) uptake — tumors show high uptake (bright spots)[25]
- SUV (Standardized Uptake Value) is the normalized intensity unit: `SUV = activity_concentration / (injected_dose / body_weight)`[26][25]
- MIP-PET images are typically 2D grayscale PNG/TIFF or DICOM slices
- High SUV areas (SUV > 2.5) are typically tumors or inflammation[27]

```
import numpy as np
import torch
from PIL import Image
import pydicom  # for DICOM files

def load_mip_pet(filepath):
    """Load and preprocess a MIP-PET image."""
    ext = filepath.lower().split('.')[-1]

    if ext in ['png', 'jpg', 'tiff', 'tif']:
        img = np.array(Image.open(filepath).convert('L'), dtype=np.float32)
    elif ext == 'dcm':
        ds = pydicom.dcmread(filepath)
        img = ds.pixel_array.astype(np.float32)
        # Apply DICOM rescale
        slope = getattr(ds, 'RescaleSlope', 1.0)
        intercept = getattr(ds, 'RescaleIntercept', 0.0)
        img = img * slope + intercept
```

```
    else:
        raise ValueError(f"Unsupported format: {ext}")

    return img

def preprocess_pet(img, target_size=(512, 512)):
    """Standard preprocessing for PET MIP images."""
    # 1. Resize
    from PIL import Image as PILImage
    pil_img = PILImage.fromarray(img.astype(np.uint8) if img.max() <= 255 els
    pil_img = pil_img.resize(target_size, PILImage.BILINEAR)
    img = np.array(pil_img, dtype=np.float32)

    # 2. Normalize to [0, 1]
    img_min, img_max = img.min(), img.max()
    if img_max > img_min:
        img = (img - img_min) / (img_max - img_min)

    # 3. Add channel dimension: (1, H, W)
    img = img[np.newaxis, ...]

    return img

def to_tensor(img_array):
    """Convert numpy (C, H, W) to torch tensor."""
    return torch.from_numpy(img_array).unsqueeze(0)  # (1, C, H, W)
```

4.2 U-Net Architecture

U-Net uses an encoder-decoder structure with **skip connections** that pass high-resolution feature maps from the encoder directly to the decoder. This preserves spatial detail crucial for segmentation — the encoder captures "what" (semantics), the skip connections preserve "where" (location).[28][29]

```
Input (1, 512, 512)
  |
```

```
Encoder:
  ├─ Block1: Conv→BN→ReLU → (64, 512, 512) ──────────────────────────┐ s
  ├─ MaxPool → (64, 256, 256)                          │
  ├─ Block2: Conv→BN→ReLU → (128, 256, 256) ──────────────────┐    │ skip
  ├─ MaxPool → (128, 128, 128)                       │    │
  ├─ Block3: Conv→BN→ReLU → (256, 128, 128) ─────────────┐    │    │ skip
  ├─ MaxPool → (256, 64, 64)                     │    │    │
  └─ Bottleneck: Conv → (512, 64, 64)                │    │    │
                                          │    │    │
Decoder:                                     │    │    │
  ├─ Upsample → Concat with skip3 → Conv → (256, 128, 128) ◀──┘
  ├─ Upsample → Concat with skip2 → Conv → (128, 256, 256) ◀────┘
  ├─ Upsample → Concat with skip1 → Conv → (64, 512, 512) ◀──────┘
  └─ 1×1 Conv → (1, 512, 512)  [segmentation mask]
```

MONAI U-Net instantiation (copy-paste ready)

```python
# pip install monai torch torchvision
import torch
import torch.nn as nn
from monai.networks.nets import UNet
from monai.networks.layers import Norm

def create_unet_2d(in_channels=1, out_channels=1):
    """2D U-Net for MIP-PET tumor segmentation."""
    model = UNet(
        spatial_dims=2,          # 2D for MIP images
        in_channels=in_channels,
        out_channels=out_channels,
        channels=(16, 32, 64, 128, 256),  # feature maps at each level
        strides=(2, 2, 2, 2),          # downsampling strides
        num_res_units=2,               # residual units per block
        norm=Norm.BATCH,               # batch normalization
        dropout=0.1,
    )
    return model
```

```
# Instantiate
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = create_unet_2d().to(device)
print(f"Parameters: {sum(p.numel() for p in model.parameters()):,}")
```

4.3 Loss Functions: What to Use

For tumor segmentation (highly imbalanced: small bright spots in large dark background):[30][31][32]

Loss	Handles Imbalance	Stability	Recommendation
BCE (Binary Cross Entropy)	Poor	High	Don't use alone for tumors
Dice Loss	Good	Medium	Good default
BCE + Dice (combined)	Best	High	**Use this**
Focal + Dice	Best	Medium	Use if combined not enough

```
from monai.losses import DiceLoss, DiceCELoss, FocalLoss
import torch.nn as nn

# ── Recommended: DiceCE (Dice + Cross Entropy combined) ──
criterion = DiceCELoss(
    sigmoid=True,    # apply sigmoid to logits (for binary output)
    lambda_dice=0.5, # weight for Dice term
    lambda_ce=0.5,   # weight for CE term
)

# ── Pure Dice (simpler, also good) ──
dice_loss = DiceLoss(sigmoid=True)

# ── Manual DiceBCE ──
```

```python
def dice_bce_loss(pred, target, smooth=1.0):
    pred = torch.sigmoid(pred)

    # Dice
    intersection = (pred * target).sum(dim=(2,3))
    dice = 1 - (2 * intersection + smooth) / (pred.sum(dim=(2,3)) + target.sum(di

    # BCE
    bce = nn.functional.binary_cross_entropy(pred, target, reduction='mean')

    return dice.mean() + bce

# Training loop
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=

def train_epoch(model, loader, optimizer, criterion):
    model.train()
    total_loss = 0
    for batch in loader:
        images = batch['image'].to(device)  # (B, 1, H, W)
        labels = batch['label'].to(device)  # (B, 1, H, W)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()

        # Gradient clipping (important for stability)
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

        optimizer.step()
        total_loss += loss.item()

    scheduler.step()
    return total_loss / len(loader)
```

4.4 MONAI Transforms and Data Pipeline

```
from monai.transforms import (
    Compose, LoadImaged, EnsureChannelFirstd,
    ScaleIntensityd, RandFlipd, RandRotated,
    RandZoomd, RandGaussianNoised, ToTensord,
    Resized, NormalizeIntensityd
)
from monai.data import Dataset, DataLoader

# Training transforms (with augmentation)
train_transforms = Compose([
    LoadImaged(keys=["image", "label"]),
    EnsureChannelFirstd(keys=["image", "label"]),
    Resized(keys=["image", "label"], spatial_size=(512, 512)),
    ScaleIntensityd(keys=["image"]),        # scale to [0, 1]
    RandFlipd(keys=["image", "label"], prob=0.5, spatial_axis=0),
    RandFlipd(keys=["image", "label"], prob=0.5, spatial_axis=1),
    RandRotated(keys=["image", "label"], prob=0.3, range_x=0.3),
    RandZoomd(keys=["image", "label"], prob=0.3, min_zoom=0.9, max_zoom=
    RandGaussianNoised(keys=["image"], prob=0.2, std=0.01),
    ToTensord(keys=["image", "label"]),
])

# Validation transforms (no augmentation)
val_transforms = Compose([
    LoadImaged(keys=["image", "label"]),
    EnsureChannelFirstd(keys=["image", "label"]),
    Resized(keys=["image", "label"], spatial_size=(512, 512)),
    ScaleIntensityd(keys=["image"]),
    ToTensord(keys=["image", "label"]),
])

# Dataset setup
# data_list = [{"image": "img1.png", "label": "mask1.png"}, ...]
```

```
train_ds = Dataset(data=train_data_list, transform=train_transforms)
train_loader = DataLoader(train_ds, batch_size=8, shuffle=True, num_worker
```

4.5 Inference Under 10 Seconds

For 2D MIP images, inference is straightforward and very fast:[33]

```
from monai.inferers import SlidingWindowInferer
import time

def predict(model, image_path, device):
    """Full inference pipeline — should be <1s for 2D MIP."""
    t0 = time.time()

    # 1. Load and preprocess
    img = preprocess_pet(load_mip_pet(image_path), target_size=(512, 512))
    img_tensor = torch.from_numpy(img).unsqueeze(0).to(device)  # (1, 1, 512,

    # 2. Inference
    model.eval()
    with torch.no_grad():
        if img_tensor.shape[-1] <= 512:
            # Direct inference for 2D MIP (very fast)
            output = model(img_tensor)
        else:
            # Sliding window for large images
            inferer = SlidingWindowInferer(
                roi_size=(512, 512),
                sw_batch_size=4,
                overlap=0.25,
            )
            output = inferer(img_tensor, model)

    # 3. Post-process
    pred = torch.sigmoid(output).squeeze().cpu().numpy()
    mask = (pred > 0.5).astype(np.uint8)
```

```
    print(f"Inference time: {(time.time()-t0)*1000:.1f}ms")
    return mask

# Speed optimization: compile with torch.compile (PyTorch 2.0+)
model = torch.compile(model)  # 20-40% speedup on GPU
```

Speed checklist for < 10s inference

- [ ] Use `torch.no_grad()` during inference
- [ ] Use GPU if available (`cuda:0`)
- [ ] Use `torch.float16` (half precision): `model.half()` + `img_tensor.half()` — 2x speedup
- [ ] Use `torch.compile(model)` (PyTorch 2.0+) for first-call compilation speedup
- [ ] Resize to 512×512 (not 1024×1024) for faster inference
- [ ] Batch multiple images together if predicting on a dataset

---

4.6 Evaluation Metrics

```
from monai.metrics import DiceMetric, HausdorffDistanceMetric
import torch

# Setup metrics
dice_metric = DiceMetric(include_background=False, reduction="mean")
hausdorff_metric = HausdorffDistanceMetric(include_background=False, per

def evaluate(model, val_loader, device):
    model.eval()
    dice_metric.reset()

    with torch.no_grad():
        for batch in val_loader:
            images = batch['image'].to(device)
            labels = batch['label'].to(device)

            outputs = torch.sigmoid(model(images))
            outputs_binary = (outputs > 0.5).float()
```

```
        dice_metric(y_pred=outputs_binary, y=labels)

    mean_dice = dice_metric.aggregate().item()
    dice_metric.reset()
    return mean_dice

# Manual IoU calculation
def iou_score(pred_mask, true_mask):
    intersection = np.logical_and(pred_mask, true_mask).sum()
    union = np.logical_or(pred_mask, true_mask).sum()
    return intersection / (union + 1e-8)

# Dice coefficient
def dice_score(pred_mask, true_mask):
    intersection = (pred_mask * true_mask).sum()
    return 2 * intersection / (pred_mask.sum() + true_mask.sum() + 1e-8)
```

Common Mistakes in Medical Image Segmentation

- **Forgetting `EnsureChannelFirst`**: MONAI expects (C, H, W); raw images are often (H, W)
- **Wrong loss sigmoid**: If using `DiceLoss(sigmoid=True)`, don't apply sigmoid to output before passing — the loss does it internally
- **Not handling empty masks**: Some PET images may have no tumor — ensure your pipeline handles all-zero labels
- **Overfitting on small dataset**: Use augmentation aggressively (flip, rotate, zoom, noise)
- **Inference on wrong dtype**: Mix of float16/float32 causes errors — keep consistent

---

SECTION 5: PRACTICAL COMPETITION STRATEGY

5.1 How Top Teams Approach 4-Day AI Competitions[34][35]

Day 1 (March 19): Understand + Baseline

- Read ALL challenge specs before writing code
- Get a dummy submission working for EVERY challenge (even if it outputs random answers)
- Understand the scoring functions — optimize what actually matters
- Grocery Bot is live now — spend 2-3 hours getting it working first

Day 2 (March 20): Foundation

- Grocery Bot: collision avoidance + basic order batching
- RAG: get end-to-end pipeline working (embed → retrieve → answer)
- Race Car: rule-based baseline
- Tumor: load data, simple U-Net training started

Day 3 (March 21): Improve

- Focus on whichever challenge has the highest delta between your score and top scores
- RAG: tune embedding model, test threshold for true/false
- Tumor: train overnight, evaluate in morning
- Bot: scale to hard/expert difficulty

Day 4 (March 22): Polish + Submit

- Fix edge cases (crashes, empty results, timeouts)
- Prioritize reliable over perfect — a system that never crashes beats one that crashes 10% of the time
- Keep a working submission ready at all times

5.2 Using LLMs for Rapid Prototyping ("Vibe Coding")

Effective prompting for competition code

```
You are a senior Python engineer competing in an AI hackathon.
Write production-quality code for: [SPECIFIC TASK]

Requirements:
- Must run within [TIME LIMIT]
- Input format: [describe]
- Output format: [describe]
- Use: [specific libraries]
- Include error handling for: [edge cases]
```

Best practices:

- Ask for one function at a time — don't ask for the entire system
- Always specify input/output format with an example
- Ask for `# type: ignore` free code — specify Python version
- Use "make it handle X edge case" as follow-up, not "rewrite everything"
- Paste actual errors into context: "I get this error: [traceback] — fix it"

Which model for what

Task	Use
Write boilerplate (WebSocket loop, FAISS setup)	Claude Sonnet / GPT-4o
Debug tricky async/MONAI errors	Claude Opus
Quick code snippets	Claude Haiku / GPT-4o-mini
Architecture decisions	Claude Opus / o3

5.3 Challenge Prioritization

Points potential ranking:

1. **Grocery Bot** (live now — accumulating points): Highest urgency. Get 5+ bots working with order completion logic → massive score advantage
2. **RAG Healthcare**: Binary true/false + topic classification — clear evaluation metric, solvable in hours
3. **Tumor Segmentation**: Dice score optimization — can reach 0.7+ with standard U-Net in a few hours
4. **Race Car**: Time-in-seconds without crashing — PID baseline gets you far fast

Recommended time allocation (4 days × ~12 hours = 48 hours):

Challenge	Hours
Grocery Bot	15 hours
RAG Healthcare	12 hours
Tumor Segmentation	12 hours
Race Car	6 hours
Setup / debugging / rest	3 hours

5.4 Leaderboard Strategy

- **Submit early**: Even a bad submission establishes your baseline and confirms the submission pipeline works
- **Submit often**: In 4-day competitions, leaderboard gaps are visible — use them to prioritize
- **Don't over-fit to leaderboard**: If you see a huge gap, investigate if you're solving the right problem
- **Keep a backup**: Always maintain a previous working submission; never delete code

---

5.5 Python Environment Setup

```
# Create isolated environment per challenge
python -m venv nm_ai_bot
source nm_ai_bot/bin/activate  # Linux/Mac
nm_ai_bot\Scripts\activate     # Windows

# Core installs
pip install websockets asyncio numpy scipy

# Challenge 2 (Race Car)
pip install gymnasium stable-baselines3 torch

# Challenge 3 (RAG)
pip install sentence-transformers faiss-cpu transformers openai

# Challenge 4 (Tumor)
pip install monai torch torchvision pydicom Pillow
```

Jupyter vs. scripts:

- Use **Jupyter** for exploratory data analysis and model debugging (tumor segmentation)
- Use **scripts (.py)** for long-running processes (bot, training loops)
- Keep a `utils.py` with shared helpers

5.6 Debugging Async WebSocket Code Quickly

```
import logging
# Enable debug logging for websockets
logging.basicConfig(level=logging.DEBUG)
logging.getLogger("websockets").setLevel(logging.INFO)  # reduce noise

# Add timing to diagnose slow compute
import time
```

```python
async def compute_action_timed(state):
    t = time.perf_counter()
    action = compute_action(state)
    elapsed = time.perf_counter() - t
    if elapsed > 1.0:
        print(f"WARNING: compute_action took {elapsed:.2f}s")
    return action

# Test without WebSocket (unit test your logic)
def test_astar():
    grid = [[True]*10 for _ in range(10)]
    grid[5][5] = False  # wall
    path = astar(grid, (0,0), (9,9))
    assert len(path) > 0, "No path found"
    assert path == (0,0)
    assert path[-1] == (9,9)
    print(f"Path length: {len(path)}")

# Run unit tests before connecting
test_astar()
```

Common async bugs:

- `RuntimeError: no running event loop` → wrap in `asyncio.run(main())`
- `TypeError: object bool can't be used in 'await'` → missing `async def` somewhere
- Bot not responding → `await ws.send(...)` is inside a regular (non-async) function

---

5.7 Version Control During Competition

```python
# Simple git workflow for competition
git init
git add .
git commit -m "initial working submission"
```

```
# Every time you improve a challenge
git add challenge1_bot.py
git commit -m "bot: add order batching, score +20%"

# If something breaks, go back
git stash         # save current broken state
git checkout HEAD  # restore last working version
git stash pop     # reapply your changes on top

# Tag your best submissions
git tag -a "bot-v2-best" -m "Grocery Bot v2 - scoring 450/round"
```

SECTION 6: QUICK REFERENCE

6.1 A* Pseudocode (Copy-Paste Ready)

```
import heapq
from collections import defaultdict

def astar(grid, start, goal):
    """
    grid: 2D list, True=passable False=wall
    Returns path as list of (row, col)
    """
    h = lambda n: abs(n-goal) + abs(n[1]-goal[1])
    open_q = [(h(start), h(start), 0, start)]
    g = defaultdict(lambda: float('inf'))
    g[start] = 0
    came_from = {}
    closed = set()

    while open_q:
        _, _, cost, node = heapq.heappop(open_q)
        if node in closed: continue
        closed.add(node)
```

```
        if node == goal:
            path = []
            while node in came_from:
                path.append(node); node = came_from[node]
            return [start] + path[::-1]
        r, c = node
        for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
            nb = (r+dr, c+dc)
            if not (0<=nben(grid) and 0<=nb[1]en(grid)): continue
            if not grid[nb][nb[1]]: continue
            ng = cost + 1
            if ng < g[nb]:
                g[nb] = ng; came_from[nb] = node
                f = ng + h(nb)
                heapq.heappush(open_q, (f, h(nb), ng, nb))
    return []
```

6.2 FAISS Index Setup (Copy-Paste Ready)

```
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer

# --- BUILD INDEX (once, offline) ---
model = SentenceTransformer('all-MiniLM-L6-v2')

texts = ["chunk 1...", "chunk 2...", ...]  # your corpus
embs = model.encode(texts, normalize_embeddings=True).astype(np.float32)

dim = embs.shape[1]  # 384 for MiniLM-L6-v2
index = faiss.IndexFlatIP(dim)  # Inner product = cosine sim (after normaliza
index.add(embs)
faiss.write_index(index, "index.faiss")

# Save metadata mapping
import json
```

```
with open("chunks.json", "w") as f:
    json.dump(texts, f)

# --- LOAD & QUERY (online) ---
index = faiss.read_index("index.faiss")
with open("chunks.json") as f:
    chunks = json.load(f)

def query(text, k=5):
    q_emb = model.encode([text], normalize_embeddings=True).astype(np.floa
    scores, indices = index.search(q_emb, k)
    return [(chunks[i], float(scores[j])) for j, i in enumerate(indices) if i >= 0]

# Usage
results = query("What is the treatment for myocardial infarction?")
for chunk, score in results:
    print(f"[{score:.3f}] {chunk[:100]}")
```

6.3 MONAI U-Net Instantiation (Copy-Paste Ready)

```
import torch
from monai.networks.nets import UNet
from monai.networks.layers import Norm
from monai.losses import DiceCELoss
from monai.metrics import DiceMetric

# --- MODEL ---
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = UNet(
    spatial_dims=2,
    in_channels=1,
    out_channels=1,
    channels=(16, 32, 64, 128, 256),
    strides=(2, 2, 2, 2),
```

```
    num_res_units=2,
    norm=Norm.BATCH,
).to(device)

# --- LOSS & OPTIMIZER ---
loss_fn = DiceCELoss(sigmoid=True)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-4)

# --- INFERENCE ---
def predict(model, img_tensor):
    """img_tensor: torch.float32, shape (1, 1, H, W)"""
    model.eval()
    with torch.no_grad():
        logits = model(img_tensor.to(device))
        prob = torch.sigmoid(logits)
        mask = (prob > 0.5).squeeze().cpu().numpy().astype('uint8')
    return mask

# --- SAVE / LOAD ---
torch.save(model.state_dict(), "unet_tumor.pth")
model.load_state_dict(torch.load("unet_tumor.pth", map_location=device))
```

6.4 WebSocket Async Loop Template (Copy-Paste Ready)

```
import asyncio
import json
import websockets
from websockets.exceptions import ConnectionClosed

WS_URL = "ws://localhost:8765"

async def handle_message(state: dict) -> dict:
    """Override with your game logic."""
    return {"action": "wait"}
```

```python
async def game_client():
    while True:  # Outer loop: reconnect on disconnect
        try:
            async with websockets.connect(
                WS_URL,
                ping_interval=20,
                ping_timeout=10,
            ) as ws:
                print("Connected!")
                async for raw in ws:
                    try:
                        state = json.loads(raw)
                        response = await asyncio.wait_for(
                            handle_message(state),
                            timeout=1.8
                        )
                        await ws.send(json.dumps(response))
                    except asyncio.TimeoutError:
                        await ws.send(json.dumps({"action": "wait"}))
                    except Exception as e:
                        print(f"Handler error: {e}")
        except ConnectionClosed:
            print("Disconnected — reconnecting in 1s...")
            await asyncio.sleep(1)
        except OSError:
            print("Network error — retrying in 2s...")
            await asyncio.sleep(2)

if __name__ == "__main__":
    asyncio.run(game_client())
```

6.5 Key Python Libraries: Install Commands

```python
# ── CHALLENGE 1: Grocery Bot ──
pip install websockets asyncio numpy scipy
```

```
# ── CHALLENGE 2: Race Car ──
pip install gymnasium stable-baselines3 torch torchvision
# For GPU training:
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu

# ── CHALLENGE 3: RAG Healthcare ──
pip install sentence-transformers faiss-cpu transformers openai
# For GPU FAISS:
# pip install faiss-gpu  (requires CUDA)

# ── CHALLENGE 4: Tumor Segmentation ──
pip install monai torch torchvision pydicom Pillow nibabel
# Full MONAI with extras:
pip install "monai[all]"

# ── GENERAL UTILITIES ──
pip install numpy pandas matplotlib scikit-learn tqdm jupyter
```

Library Quick Reference

Library	Purpose	Import
`websockets`	WebSocket client/server	`import websockets`
`asyncio`	Async event loop	`import asyncio`
`heapq`	Priority queue for A*	`import heapq`
`sentence-transformers`	Text embeddings	`from sentence_transformers import SentenceTransformer`
`faiss-cpu`	Vector similarity search	`import faiss`
`transformers`	NLI/zero-shot	`from transformers import pipeline`
`monai`	Medical image AI	`from monai.networks.nets import UNet`
`stable-baselines3`	RL algorithms	`from stable_baselines3 import PPO, SAC`
`gymnasium`	RL environments	`import gymnasium as gym`
`torch`	Deep learning	`import torch, torch.nn as nn`
`scipy`	Hungarian algorithm	`from scipy.optimize import linear_sum_assignment`
`numpy`	Array operations	`import numpy as np`
`pydicom`	DICOM file loading	`import pydicom`

Good luck, Sander! Start with the Grocery Bot (it's already live), get a WebSocket connection working first, then layer in A and order logic. For the RAG challenge, the MiniLM + FAISS pipeline is your fastest path to a working system. For tumor segmentation, MONAI makes the heavy lifting manageable.*

References

1. A* Pathfinding Algorithm Tutorial: Complete Implementation ... - Master A* pathfinding algorithm with complete C# and Unity implementation. Learn heuristics, optimiz...
2. Cooperative Pathfinding - by D Silver · 2005 · Cited by 1218 — The reservation table represents the agents' shared knowl- edge...
3. Deadlock-Free Method for Multi-Agent Pickup and Delivery ... - by Y Fujitani · 2022 · Cited by 8 — This paper proposes a control method for the multi-agent pickup ...
4. Standby-Based Deadlock Avoidance Method for Multi- ... - by T Yamauchi · Cited by 29 — [21] proposed priority inheritance with backtracking (PIBT) in which t...
5. Priority inheritance with backtracking for iterative multi- ... - by K Okumura · 2022 · Cited by 264 — To avoid agents getting stuck waiting, priority inheritance is ...
6. Order Batching: What It Is, Common Methods, and Benefits - Learn how an order batching strategy works, its strategic advantages, and how to apply it to maximiz...
7. How to Reconnect Python Websockets After a Timeout in ... - Learn how to handle websocket timeouts and reconnect in Python using asyncio, with tips to manage ...
8. How to Build WebSocket Clients in Python - Best Practices Summary · Always use async context managers - They handle connection cleanup automati...
9. [2001.05270] Continuous-action Reinforcement Learning ... - by MS Holubar · 2020 · Cited by 21 — In this paper, a novel racing environment for OpenAI Gym is int...
10. PID Controller in Robotics—A Practical Deep Dive with Python ... - Learn how a PID controller works with Python and C++ examples. Understand tuning, visualization, and...
11. Solving Gymnasium's Car Racing with Reinforcement Learning - In this blog post, we dive into the exciting process of solving the

Car Racing environment using thr...

12. Reinforcement Learning CarRacing environment using PPO - This repository provides one of the simplest solutions for the CarRacing-v3 environment from Gymnasi...
13. Adapting Gymnasium to Find DeepRacer Racing Lines - Adapting Gymnasium to Find DeepRacer Racing Lines. Train a PPO agent from Stable Baselines3 to maste...
14. Implement a WebSocket Server in PyGame to control objects via a HTML WebSocket Client - General idea I successfully configured a raspberry pi as an access point such that I can connect via...
15. RAG Pipeline Deep Dive: Ingestion, Chunking, Embedding, and ... - Retrieval-Augmented Generation (RAG) has become the default approach for providing context and memor...
16. all-MiniLM-L6-v2 by sentence-transformers - Q4KM.ai - The all-MiniLM-L6-v2 model is a compact, high-performance sentence encoder built on the
17. Unlock Powerful Embeddings Using all-MiniLM-L6-v2 - The all-MiniLM-L6-v2 model is an efficient tool for creating sentence embeddings. It converts senten...
18. Guidelines to choose an index · facebookresearch/faiss Wiki - GitHub - A library for efficient similarity search and clustering of dense vectors. - facebookresearch/faiss
19. Nearest Neighbor Indexes for Similarity Search - This article will explore the pros and cons of some of the most important indexes — Flat, LSH, HNSW,...
20. cross-encoder/nli-distilroberta-base - Hugging Face - We’re on a journey to advance and democratize artificial intelligence through open source and open s...
21. Sentence Embeddings. Cross-encoders and Re-ranking - Deep Dive into Cross-encoders and Re-ranking
22. Zero-Shot Classification: Building Models That Generalize ... - By leveraging semantic relationships and transferable knowledge, zero-shot classification enables mo...
23. MIP-Based Tumor Segmentation: A Radiologist-Inspired ... - Although deep learning has been applied to PET-CT for lesion detection and disease staging, most mod...
24. [Literature Review] Segmentation-Free Outcome Prediction ... - The method leverages deep learning (DL) for feature extraction

from multi-angle maximum intensity pr...
25. PET/CT Standardized Uptake Values (SUVs) in Clinical ... - PMC - by PE Kinahan · 2010 · Cited by 836 — The use of standardized uptake values (SUVs) is now common pla...
26. Standardized uptake value - Wikipedia
27. Standardized uptake value (SUV) numbers on PET scans - The measurement of brightness is the SUV number. In general, higher SUV numbers may indicate a malig...
28. The impact of U-Net architecture choices and skip ... - by A Kamath · 2025 · Cited by 4 — Our study investigates three levels of skip connection density: en...
29. U-Net-Based Medical Image Segmentation - PMC - by XX Yin · 2022 · Cited by 395 — These segmentation networks share a common feature—skip connection...
30. Unified Focal loss: Generalising Dice and cross entropy-based ... - Automatic segmentation methods are an important advancement in medical image analysis. Machine learn...
31. Generalising Dice and cross entropy-based losses to handle class ... - Automatic segmentation methods are an important advancement in medical image analysis. Machine learn...
32. Unified Focal loss: Generalising Dice and cross entropy ... - Automatic segmentation methods are an important advancement in medical image analysis. Machine learn...
33. Inference methods#
34. Winning Strategies for Kaggle Competitions | PDF - Additional tips include using good code practices like commenting and version control, reusing code ...
35. Things I Learned by Participating in GenAI Hackathons ... - Things I Learned by Participating in GenAI Hackathons Over the Past 6 Months · 1. Every idea starts ...